Université des sciences et de la technologie d'Oran – USTO-MB
Faculté des Mathématiques et de l'Informatique - Département d'informatique
3ème Année LMD – S6 – SI & ISIL

Cours 6

Données semi structurées

L3 Informatique / FMI / USTO-MB / 2022-2023

Cours présenté par K. BELMABROUK

21/02/2025

6. XQUERY et les BD

6. 1. XML et données semi structurées

6. 2. XQUERY

6. XQUERY et les BD

6. 1. XML et données semi structurées

6. 2. XQUERY

6. 1. XML et données semi structurées

6.1.1. Bases de Données semi- structurées

6.1.2. Langages de requêtes

6.1.3. BD XML

6. 1. XML et données semi structurées

6.1.1. Bases de Données semi- structurées, C'est quoi?

Une base de données semi-structurée est une base de données qui permet de **stocker des données** de manière **flexible** et souvent **hiérarchique sans suivre un schéma de données rigide** comme dans les bases de données relationnelles classiques.

6. 1. XML et données semi structurées

6.1.2. Langages de requêtes

- **SQL (Structured Query Language) :** c'est le langage de requête le plus couramment utilisé pour interroger et manipuler des données dans les bases de données relationnelles.
- **XQuery :** c'est un langage de requête pour les bases de données XML, utilisé pour extraire et manipuler des données dans des documents XML.
- **SPARQL :** c'est un langage de requête pour les bases de données RDF (Resource Description Framework), utilisé pour interroger et manipuler des données dans des graphes RDF.

6. 1. XML et données semi structurées

6.1.2. Langages de requêtes (suite)

- **Gremlin :** c'est un langage de requête pour les bases de données orientées graphe, utilisé pour interroger et manipuler des données dans des graphes.
- **Cypher :** c'est un langage de requête pour la base de données orientée graphe Neo4j, utilisé pour interroger et manipuler des données dans des graphes.
- Il existe **d'autres langages** de requêtes spécifiques à certaines bases de données ou à certains types de données, tels que **MDX pour les cubes OLAP** et **DAX pour Power Pivot**.

6. 1. XML et données semi structurées

6.1.3. BD XML

- Une **base de données XML** est une base de données qui stocke des données sous forme de **documents XML hiérarchiques** et permet **d'interroger**, de **modifier** et d'extraire ces données à l'aide de **langages de requête** tels que **XQuery**.
- Un **document XML hiérarchique** est un document XML qui contient des **éléments imbriqués** les uns dans les autres de manière hiérarchique, de la même manière qu'un **arbre avec des branches**.

6. XQUERY et les BD

6. 1. XML et données semi structurées

6. 2. XQUERY

6. 2. XQUERY

XQuery permet:

- **d'extraire des informations** d'un document ou d'une collection de documents XML,
- **d'effectuer des calculs complexes** à partir des informations extraites
- de **reconstruire de nouveaux documents** ou **fragments** XML.

6. 2. XQUERY

XQuery joue par rapport aux **données XML** un **rôle similaire** à celui du langage **SQL** vis-à-vis des **données relationnelles**

6. 2. XQUERY

- **XQuery** : W3C Working Draft **02 May 2003**
- **XQuery 1.0** : la recommandation du W3C, **23 Janvier 2007**
- **XQuery** est supporté par toutes les **B.D.** importantes (**IBM**, **ORACLE**, **Microsoft**, etc.)

6. 2. XQUERY

XQuery est un langage fonctionnel (typé) :

- Une requête **XQuery** est une **composition d'expressions**
- Chaque **expression** retourne une **valeur** (ou une **erreur**)
- Les **expressions** n'ont pas d'effets de bord (mises-à-jour)
- Tout **XPath** est une **requête valide**

6. 2. XQUERY

Quelques exemples de requêtes

1. Sélectionner tous les titres des livres:

```
for $book in //book return $book/title
```

2. Sélectionner tous les auteurs des livres dont le prix est inférieur à 20 :

```
for $book in //book where $book/price < 20
return $book/author
```

6. 2. XQUERY

6.2.1. Syntaxe XQuery

6.2.2. Usages et exemples de fonctions

6. 2. XQUERY

6.2.1. Syntaxe XQuery

6.2.2. Usages et exemples de fonctions

6.2.1. Syntaxe XQuery

Il existe deux syntaxes distinctes pour XQuery :

1. la syntaxe "**naturelle**" non-XML dite aussi **FLWOR** (prononcer flower), dont le nom vient des cinq clauses principales qui la composent (`for`, `let`, `where`, `order by` et `return`) ;
2. la syntaxe **Xquery**, dans laquelle une requête est un document XML. De ce fait, elle est beaucoup plus verbeuse et moins lisible que la précédente et est destinée à des manipulations formelles par des programmes (éventuellement eux-mêmes écrits en XQuery).

6.2.1. Syntaxe XQuery

- Toutes les valeurs manipulées par **XQuery** sont des séquences d'items ⇒ mise à plat des séquences imbriquées.
- Les items se répartissent en deux grands groupes :
  1. **Les nœuds XML**, qui eux-mêmes sont: document, élément, attribut, texte, commentaire, processing-instruction.
  2. **Les types de base** empruntés aux Schémas XML (entiers, décimaux, flottants, chaines de caractères et dérivés,…)
- Les types de bases peuvent être **étendus** par l'im**portation de Schémas** (fonctionnalité optionnelle).

6.2.1. Syntaxe XQuery

6.2.1.1. Expressions XQuery

6.2.1.2. Fonctions et opérations

6.2.1.3. Modules optionnels

6.2.1.4. Extensions

6.2.1. Syntaxe XQuery

6.2.1.1. Expressions XQuery

6.2.1.2. Fonctions et opérations

6.2.1.3. Modules optionnels

6.2.1.4. Extensions

6.2.1. Syntaxe XQuery

6.2.1.1. Expressions Xquery

Expressions simples

- Valeurs **atomiques** :
  - `13` est une requête qui retourne l'entier **13**
  - `"Bonjour"` est une requête qui retourne la chaîne de caractères **"Bonjour"**
- Valeurs **construites** : `true(), date("6-11-2007")`

Expressions complexes

- Expressions de **chemins** : `livre//section`
- Expressions **FLWR** (**F**or-**L**et-**W**here-**R**eturn) :**FOR** $<var> in <forest> [, $<var> in <forest>]... // itération **LET** $<var> := <subtree> // assignation **WHERE** <condition> // élagage **RETURN** <result> // construction
- **Tests :** `if-then-return-else-return`

6.2.1. Syntaxe XQuery

6.2.1.1. Expressions XQuery

6.2.1.2. Fonctions et opérations

6.2.1.3. Modules optionnels

6.2.1.4. Extensions

6.2.1. Syntaxe XQuery

6.2.1.2. Fonctions et opérations

Fonctions

- Fonctions **prédéfinies** : doc("data/books.xml")//book/author
- Fonctions **utilisateurs**

Opérateurs

- Opérateurs **arithmétiques**
- Opérations sur **les séquences** (**concaténation**, **union**, **intersection**, **différence**)

6.2.1. Syntaxe XQuery

6.2.1.1. Expressions XQuery

6.2.1.2. Fonctions et opérations

6.2.1.3. Modules optionnels

6.2.1.4. Extensions

6.2.1. Syntaxe XQuery

6.2.1.3. Modules optionnels :

Le module optionnel *Full Axis* (Axes étendus) permet de bénéficier des "axes" de requête *ancestor, ancestor-or-self, following, following-sibling, preceding*, et *preceding-sibling* dans une expression XPath.

Les modules XQuery permettent d'importer des bibliothèques de fonctions ou de variables XQuery dans un programme XQuery.

Le module optionnel *Schema Import* permet de spécifier les schémas XML auxquels répondent les données manipulées, permettant ainsi d'inférer les types de certaines expressions, et éventuellement d'optimiser les requêtes.

Le module optionnel *Schema Validation* permet d'utiliser le mécanisme de validation de fragments XML par rapport à des schémas.

Le module optionnel "Typage Statique" supporte des vérifications de type plus poussées.

6.2.1. Syntaxe XQuery

6.2.1.1. Expressions XQuery

6.2.1.2. Fonctions et opérations

6.2.1.3. Modules optionnels

6.2.1.4. Extensions

6.2.1. Syntaxe XQuery

6.2.1.4. Extensions

- XQuery Update est une norme en cours d'élaboration (norme préliminaire de 2010) étendant XQuery par des instructions de modification de nœuds XML : insert node, delete node, rename node, replace node, replace value, copy/modify.
- XQuery Full-Text est une extension en cours d'élaboration (norme préliminaire en 2008) spécifiant la recherche textuelle intégrée à XQuery. Elle permet la recherche contextuelle de mots et de phrases.
- XQuery Scripting, en cours d'élaboration (Working Draft d'avril 2010), modifie le modèle de programmation pour ajouter des instructions impératives (blocs séquentiels, while, exit) garantissant l'ordre d'exécution.

6. 2. XQUERY

6.2.1. Syntaxe XQuery

6.2.2. Usages et exemples de fonctions

6. 2. XQUERY

(Exemples de fonctions)

Nous utiliserons le fichier books.xml dans les exemples suivants:

Sélection de noeuds avec FLWOR
L'expression XPath

```
doc('books.xml')/bookstore/book[price>30]/title
```

est équivalente à l'expression FLWOR suivante

```
for $x in doc("books.xml")/bookstore/book
  where $x/price>30
  return $x/title
```

Le résultat :

```
<title lang="en">XQuery Kick Start</title>
<title lang="en">Learning XML</title>
```

Il est possible de trier les éléments avec FLWOR

```
for $x in doc("books.xml")/bookstore/book
  where $x/price>30
  order by $x/title
  return $x/title
```

```
▼<bookstore>
  ▼<book category="COOKING">
      <title lang="en">Everyday Italian</title>
      <author>Giada De Laurentiis</author>
      <year>2005</year>
      <price>30.00</price>
    </book>
  ▼<book category="CHILDREN">
      <title lang="en">Harry Potter</title>
      <author>J K. Rowling</author>
      <year>2005</year>
      <price>29.99</price>
    </book>
  ▼<book category="WEB">
      <title lang="en">XQuery Kick Start</title>
      <author>James McGovern</author>
      <author>Per Bothner</author>
      <author>Kurt Cagle</author>
      <author>James Linn</author>
      <author>Vaidyanathan Nagarajan</author>
      <year>2003</year>
      <price>49.99</price>
    </book>
  ▼<book category="WEB">
      <title lang="en">Learning XML</title>
      <author>Erik T. Ray</author>
      <year>2003</year>
      <price>39.95</price>
    </book>
</bookstore>
```

6. 2. XQUERY

(Exemples de fonctions)

Création de nœuds

```
<ul>
{
for $x in doc("books.xml")/bookstore/book/title
  order by $x
  return <li>{$x}</li>
}
</ul>
```

Le résultat :

```
<ul>
<li><title lang="en">Everyday Italian</title></li>
<li><title lang="en">Harry Potter</title></li>
<li><title lang="en">Learning XML</title></li>
<li><title lang="en">XQuery Kick Start</title></li>
</ul>
```

```
▼<bookstore>
  ▼<book category="COOKING">
      <title lang="en">Everyday Italian</title>
      <author>Giada De Laurentiis</author>
      <year>2005</year>
      <price>30.00</price>
    </book>
  ▼<book category="CHILDREN">
      <title lang="en">Harry Potter</title>
      <author>J K. Rowling</author>
      <year>2005</year>
      <price>29.99</price>
    </book>
  ▼<book category="WEB">
      <title lang="en">XQuery Kick Start</title>
      <author>James McGovern</author>
      <author>Per Bothner</author>
      <author>Kurt Cagle</author>
      <author>James Linn</author>
      <author>Vaidyanathan Nagarajan</author>
      <year>2003</year>
      <price>49.99</price>
    </book>
  ▼<book category="WEB">
      <title lang="en">Learning XML</title>
      <author>Erik T. Ray</author>
      <year>2003</year>
      <price>39.95</price>
    </book>
  </bookstore>
```

6. 2. XQUERY

(Exemples de fonctions)

Expressions conditionnelles

```
for $x in doc("books.xml")/bookstore/book
return if ($x/@category="CHILDREN")
  then <child>{data($x/title)}</child>
  else <adult>{data($x/title)}</adult>
```

Le résultat :

```
<adult>Everyday Italian</adult>
<child>Harry Potter</child>
<adult>Learning XML</adult>
<adult>XQuery Kick Start</adult>
```

```
▼<bookstore>
  ▼<book category="COOKING">
      <title lang="en">Everyday Italian</title>
      <author>Giada De Laurentiis</author>
      <year>2005</year>
      <price>30.00</price>
    </book>
  ▼<book category="CHILDREN">
      <title lang="en">Harry Potter</title>
      <author>J K. Rowling</author>
      <year>2005</year>
      <price>29.99</price>
    </book>
  ▼<book category="WEB">
      <title lang="en">XQuery Kick Start</title>
      <author>James McGovern</author>
      <author>Per Bothner</author>
      <author>Kurt Cagle</author>
      <author>James Linn</author>
      <author>Vaidyanathan Nagarajan</author>
      <year>2003</year>
      <price>49.99</price>
    </book>
  ▼<book category="WEB">
      <title lang="en">Learning XML</title>
      <author>Erik T. Ray</author>
      <year>2003</year>
      <price>39.95</price>
    </book>
  </bookstore>
```

6. 2. XQUERY

Exemples de serveurs de BDXML

1. **eXist** : eXist est une base de données XML **open source** qui prend en charge le **stockage de documents XML** ainsi que la **recherche** et **l'interrogation** de ces documents en utilisant **XQuery**.
2. **MarkLogic** : MarkLogic est une base de données **NoSQL** qui prend en charge la **gestion de données XML, JSON et RDF**. MarkLogic prend en charge les langages de requête **XQuery, SQL** et **SPARQL** pour interroger les données.
3. **BaseX** : BaseX est une base de données XML **open source** qui prend en charge le **stockage** et la **récupération de données XML**, ainsi que les langages de requête **XQuery** et **XPath**.

Merci de votre attention!